



Web Development Lab

Hero Section— Practical Session Guide

15 Guided Steps • Beginner–Intermediate • Estimated Time: 50 mins

Lab Overview

In this session you will build a modern Hero Section — the most important part of any landing page. This section introduces users to your product using a headline, description, call-to-action buttons, and an image.

You will use Flexbox for layout, improve spacing, control typography, and implement interactive UI elements (hover effects).

Each step builds on the previous one so you can see real-time progress in your browser.

 **Note:** All code will be written inside your existing `index.html` file (continuing from the Navbar lab).

Prerequisites

- Completed Navbar Lab 
- Basic understanding of HTML structure
- Familiarity with Flexbox
- A web browser (Chrome recommended)

1 Add the Hero Section HTML Structure

Objective: Create the structural layout of the hero section.

Background

The Hero Section is divided into two main parts:

- Left = Text content (headline, description, buttons)
- Right = Image

We use **<section>** to group related content semantically.

Instructions

1. Below your navbar, add the Hero Section HTML

Code Snippet

```
<div class="heroSection">
  <section class="herSectionLeft">
    <h2>
      Field Documentation Made Smarter With Digital Precision.
    </h2>
    <p>
      We aim to revolutionize how construction and architecture work is
documented,
      tracked, and delivered through innovative technology.
    </p>
    <p class="buttonparagraph">
      <button>Get Started</button>
      <span>Explore Features</span>
    </p>
  </section>

  <section class="herSectionRight">
    
  </section>
</div>
```



Tip: Hero sections usually follow a text-left, image-right pattern.

2 Convert Hero Section into a Flex Container

Objective: Enable horizontal layout using Flexbox.

Background

1. **display: flex;**
 - Turns `.heroSection` into a **flex container**
 - Automatically arranges children (**left and right sections**) horizontally
2. **align-items: center;**
 - Aligns items vertically (cross-axis)
3. **justify-content: space-between;**
 - Pushes left and right sections apart

Instructions

1. Add Flexbox to `.heroSection`

Code Snippet

```
.heroSection{
  display: flex;
  align-items: center;
  justify-content: space-between;
}
```



Tip: This is the same concept used in your navbar — consistency matters.

3 Add Background and Spacing

Objective: Improve layout appearance with spacing and color.

Background

1. **background-color: #FAFAF8;**
 - Applies a soft background color
2. **margin: 0 82px;**
 - Adds spacing on left and right
3. **padding: 80px 0;**
 - Adds vertical space inside the section

Instructions

1. Add spacing and background

Code Snippet

```
.heroSection{  
  background-color: #FAFAF8;  
  margin: 0 82px;  
  padding: 80px 0;  
}
```



Tip: Margin = outside spacing, Padding = inside spacing.

4 Add Gap Between Sections

Objective: Create space between left and right sections.

🕒 4 min

Background:

Spacing and background color improve visual clarity and structure of the layout.

1. **gap: 100px;**
 - Adds space **between flex items**
 - Cleaner than using margins

Instructions

1. Add gap property

Code Snippet

```
.heroSection{
  background-color: #FAFAF8;
  margin: 0 82px;
  padding: 80px 0;
  gap: 100px;
}
```

💡 **Tip:** **gap** works in Flexbox and Grid.

5 Control Section Height

Objective: Ensure consistent vertical spacing.

🕒 4 min

Background

1. min-height: 500px;
 - Prevents section from collapsing
 - Helps with vertical centering

Instructions

1. Add minimum height

Code Snippet

```
.heroSection{
  background-color: #FAFAF8;
  margin: 0 82px;
  padding: 80px 0;
  gap: 100px;
  min-height:500px;
}
```



Tip: Hero sections usually have strong vertical presence.

6. Make Left Section Responsive

Objective: Control size and flexibility of text section.

Background

Flex properties help distribute space dynamically and improve responsiveness.

1. **flex: 1;**
 - Allows the section to grow and shrink equally
2. **max-width: 628px;**
 - Prevents text from becoming too wide

Instructions

1. Style `.herSectionLeft`

Code Snippet

```
.herSectionLeft{  
  flex: 1;  
  max-width: 628px;  
}
```



Tip: Long lines reduce readability — limit width.

7. Style the Heading

Objective: Emphasize the main message.

🕒 6 min

Background

Typography enhances readability and visual hierarchy.

1. **font-size: 56px;**
 - Large text for emphasis
2. **line-height: 1.2;**
 - Controls vertical spacing between lines
3. **margin-bottom: 24px;**
 - Adds space below heading

Instructions

1. Style `<h2>`

Code Snippet

```
.herSectionLeft h2{
  font-size: 56px;
  font-weight: bold;
  line-height: 1.2;
  margin-bottom: 24px;
  color: #333;
}
```

💡 **Tip:** Headlines should be bold and easy to scan.

8. Style the Paragraph Text

Objective: Improve readability of description text.

🕒 6 min

Background

1. **font-size: 18px;**
 - Standard readable size
2. **line-height: 1.75;**
 - Improves readability
3. **max-width: 500px;** *this is the maximum width attainable by any element, it cannot go beyond this size*
 - Prevents long lines

Instructions

1. Style paragraph

Code Snippet

```
.herSectionLeft p{  
  margin: 0 0 40px 0;  
  font-size: 18px;  
  line-height: 1.75;  
  color: #4A4A4A;  
  max-width: 500px;  
}
```



Tip: Good typography improves user experience significantly.

9. Create Button Layout

Objective: Align button and text horizontally.

Background

1. **display: flex;**
 - Places button and text side-by-side
2. **gap: 24px;**
 - Adds space between them

Instructions

1. Style `.buttonparagraph`

Code Snippet

```
.buttonparagraph{  
  display: flex;  
  align-items: center;  
  gap: 24px;  
}
```



Tip: Flexbox is ideal for small UI components like this.

10. Style the Primary Button

Objective: Create a call-to-action button.

🕒 6 min

Background

1. **background-color: #FF5733;**
 - Primary brand color
2. **cursor: pointer;**
 - Indicates interactivity
3. **transition: background-color 0.3s ease;**
 - Smooth hover animation

Instructions

1. Style button

Code Snippet

```
.herSectionLeft button{
  background-color: #FF5733;
  color: #FFFFFF;
  border: none;
  border-radius: 6px;
  height: 52px;
  width: 153px;
  font-size: 16px;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.3s ease;
}
```



Tip: Buttons should stand out clearly.

11. Add Button Hover Effect

Objective: Improve interactivity.

🕒 4 min

Background

1. :hover

- Applies styles when user hovers

Instructions

1. Add hover effect

Code Snippet

```
.herSectionLeft button:hover {  
  background-color: #e64a2e;  
}
```



Tip: Micro-interactions improve UX.

12. Style Secondary Action Text

Objective: Style “Explore Features” link.

🕒 4 min

Background

- text-decoration: underline;
- Indicates clickable text

Instructions

1. Style span

Code Snippet

```
.buttonparagraph span {  
  font-size: 16px;  
  color: #666;  
  text-decoration: underline;  
  cursor: pointer;  
}
```



Tip: Secondary actions should be less prominent than buttons.

13. Style Right Section (Image Container)

Objective: Center the image properly.

Background

1. **display: flex;**
2. **justify-content: center;**
3. **align-items: center;**
 - Centers content horizontally and vertically

Instructions

1. Style `.herSectionRight`

Code Snippet

```
.herSectionRight {  
  flex: 1;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```



Tip: Nested Flexbox is very common in real-world layouts.

14. Make Image Responsive

Objective: Ensure image scales properly.

Background

1. **max-width: 100%;**
 - Prevents overflow
2. **height: auto;**
 - Maintains aspect ratio

Instructions

1. Style image

Code Snippet

```
.herSectionRight img{  
  max-width: 100%;  
  height: auto;  
  border-radius: 8px;  
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);  
}
```



Tip: Always make images responsive.

16. Final Review and Testing

Objective: Validate the complete Hero Section.

🕒 8 min

Background

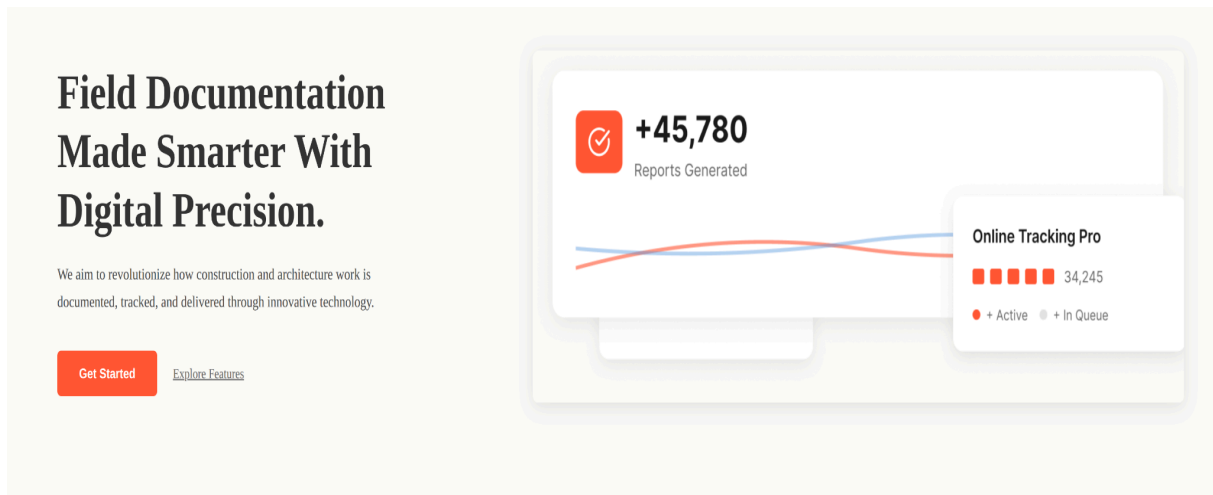
A well-built hero section should:

- Align content properly
- Maintain spacing
- Look balanced across screen sizes

Instructions

1. Refresh your browser
2. Resize the screen
3. Check spacing and alignment
4. Confirm button interactions

Final Outcome



Final Check

- No CSS errors
- Layout matches expected design